

Where Crimes are Occuring in Dallas

Rishi Natarajan

2026-04-08

I can't display the following datasets I am using on R-Studio, because it is too large, but here is the link to the dataset I am using for where dallas crimes are occuring: https://www.dallasopendata.com/Public-Safety/Police-Incidents/qv6i-rri7/about_data

API: <https://www.census.gov/data/developers.html>

This was the link I used to request an API Key: https://api.census.gov/data/key_signup.html

The main question I want to answer regarding the where is whether do crimes occur more closer or father from the highway. I am using the following libraries and an env file for my api to access census data. Finally I load the dallas crime data

```
library(dotenv) # Import env file with my api key
library(tidycensus) # Import library for census data

library(tigris) # Gets outlines for counties
```

```
## To enable caching of data, set `options(tigris_use_cache = TRUE)`
## in your R script or .Rprofile.
```

```
library(ggplot2) # Plotting map
library(dplyr) # For math for data analysis
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(sf) # Checks what touches dallas county
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(ggimage) # For interstate and highway shields
```

```
# Load the file from the parent directory  
load_dot_env(file = "../.env")
```

```
## Warning in readLines(file): incomplete final line found on '../.env'
```

```
# Pulls api key from the env file  
census_api_key(Sys.getenv("CENSUS_API_KEY"), install = TRUE, overwrite = TRUE)
```

```
## Your original .Renviron will be backed up and stored in your R HOME directory if needed.
```

```
## Your API key has been stored in your .Renviron and can be accessed by Sys.getenv("CENSUS_API_KEY").  
## To use now, restart R or run `readRenviron("~/Renviron")`
```

```
## [1] "218532ae359940fe26df7da074fed04554e1a880"
```

```
# loads table  
master_analysis <- read.csv("../Dallas_Crimes_dallas_Website.csv")
```

Here, I fetch the population data of how many people live in each zip code in dallas county. This will allow us to get an accurate proportion, and this way, the number of crimes won't get skewed towards the zip codes that have a higher population. this way we can get a more accurate answer of how safe each zip code really is.

```
# --- 2. FETCH POPULATION DATA (Using the API) ---  
# We switch to 'acs5' for 2020 to ensure ZCTA availability  
pop_data <- get_acs(  
  geography = "zcta",  
  variables = "B01003_001", # ACS variable for Total Population  
  year = 2020,  
  survey = "acs5"  
) %>%  
  # Clean the names to match your crime data  
  mutate(zip_clean = as.character(as.integer(gsub("ZCTA5 ", "", NAME)))) %>%  
  select(zip_clean, population = estimate) # ACS uses 'estimate' instead of 'value'
```

```
## Getting data from the 2016-2020 5-year ACS
```

Here, we are loading and preping the crime data. First I am getting the zip code as an integer, so this way we can get rid of any accidental decimal points. Next, I categorize whether the crime is a Violent Crime, Property Crime, or Society Crime. I do this by using key words in the “Call (911) Problem” column. Based on these key words, we can sort it into what crime actually occurs in which zip code. Finally, we filter the other columns out, because they may be non crime reporting 911 calls.

```
# --- 3. LOAD & PREP CRIME DATA ---  
# Replace your old crime_summary with this new "Categorized" version  
crime_summary <- master_analysis %>%  
  mutate(  
    zip_clean = as.character(as.integer(Zip.Code)),
```

```

# This maps the 911 call descriptions to the 3 NIBRS categories
category = case_when(
  # PERSONS (Violent)
  grepl("ASSAULT|ROBBERY|SHOOTING|STABBING|MURDER|KIDNAP", Call..911..Problem, ignore.case = TRUE) ~ "Persons",

  # PROPERTY
  grepl("THEFT|BURGLARY|VANDAL|PROPERT|FORGERY|FRAUD", Call..911..Problem, ignore.case = TRUE) ~ "Property",

  # SOCIETY (Drugs/Weapons/Order)
  grepl("DRUG|NARCOTIC|WEAPON|GUN|PROSTITUT|GAMBL", Call..911..Problem, ignore.case = TRUE) ~ "Society",

  TRUE ~ "Other"
)
) %>%
# Filter for Dallas zips and remove the "Other" category to keep the map clean
filter(grepl("^75", zip_clean), category != "Other") %>%
group_by(zip_clean, category) %>%
summarise(total_incidents = n(), .groups = "drop")

```

Here, we are organizing our zip codes so that we will only receive data for dallas county. We do this by getting the 2020 outlines for every zip code, and we filter by making sure every zip code starts with 75 to make sure it is a part of or at least close to dallas county. Next, we import all the texas highways, but then we use the filter function to be specific to dallas county. This is why some areas of highways are cut off, because parts of them are not in dallas county. (EX PGBT)

```

# --- 4. FETCH GEOMETRY (Maps & Roads) ---
dallas_zips <- tigris::zctas(year = 2020, cb = TRUE) %>%
  filter(grepl("^75", ZCTA5CE20))

```

ZCTAs can take several minutes to download. To cache the data and avoid re-downloading in future R sessions:

```
## |
```

```
tx_highways <- tigris::primary_secondary_roads(state = "TX", year = 2020)
```

```
## |
```

```

# --- 4. FETCH GEOMETRY (Maps & Roads) ---
dallas_highways <- st_intersection(tx_highways, st_union(dallas_zips)) %>%
  filter(
    # 1. Keep all Interstates
    RTTYP == "I" |
    # 2. Keep the specific Tollways
    grepl("Dall.*North.*Tollway|George.*Bush.*Tpk", FULLNAME, ignore.case = TRUE) |
    # 3. FIX: Only keep US-75, exclude US-175
    # The \\b75\\b ensures it doesn't match 175 or 750
    (RTTYP == "U" & grepl("\\b75\\b", FULLNAME))
  ) %>%
  mutate(road_type = case_when(
    RTTYP == "I" ~ "Interstate",
    grepl("\\b75\\b", FULLNAME) ~ "US Highway",
  ))

```

```
TRUE ~ "Tollway"
))
```

```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

This is where we reduce the boundaries to make sure that we are only getting the data for dallas county. Although we already made sure that our zip code started in 75, we need to make sure that our zip codes are all in dallas county. The reason the step earlier of making sure all our zip codes started with 75, is that it can now take less time to cut down our zip codes to make sure that it is in dallas county. This will make the processing time a bit quicker. Finally, we have the highway shields so that I can label them on the maps.

```
sf_use_s2(FALSE)
```

```
## Spherical geometry (s2) switched off
```

```
# --- 5. THE DALLAS COUNTY FILTER ---
# 1. Get the official Dallas County Boundary (FIPS 113 for Dallas, TX)
dallas_county_outline <- counties(state = "TX", cb = TRUE) %>%
  filter(NAME == "Dallas")
```

```
## Retrieving data for the year 2024
```

```
## |
```

```
# 2. Filter Zip Codes to ONLY those that touch Dallas County
map_base <- dallas_zips %>%
  mutate(match_id = as.character(as.integer(ZCTA5CE20))) %>%
  # This is the "Cookie Cutter" step
  st_filter(dallas_county_outline, .predicate = st_intersects) %>%
  left_join(pop_data, by = c("match_id" = "zip_clean"))
```

```
## although coordinates are longitude/latitude, st_intersects assumes that they
## are planar
```

```
# --- 5.5 STABLE HIGHWAY ICONS (Local Files) ---
highway_shields <- data.frame(
  name = c("I-635", "I-35E", "I-30", "US-75", "DNT", "PGBT", "I-45", "I-20"),
  long = c(-96.80, -96.87, -96.72, -96.73, -96.82, -96.75, -96.75, -96.78),
  lat = c(32.925, 32.82, 32.795, 32.96, 32.85, 33.00, 32.69, 32.65),
  # Matching your screenshot names exactly
  image = c(
    "I-635.svg.png",
    "I-35E_(TX).png", # If this fails, rename the file to .png manually
    "I-30.svg.png",
    "US_75.svg.png",
    "Toll_Texas_Dallas_North_Tollway.svg.png",
    "Toll_Texas_President_George_Bush_Turnpike.svg.png", # Same here
    "I-45_(TX).svg.png", # And here
    "I-20.svg.png"
  )
)
```

Finally, here we are plotting our data. First, I have a global theme to keep the graphs looking the same. next, I have a mapping function that helps keep the three maps with the same zoom, highway layers, and crime calculations. After that, we graph our three maps and highlight the zip codes accordingly. However, we have a problem, because for our property crime map, there are some areas that show an extremely high crime density. This is because there are more crimes that occur than residents that actually live in that zip code. This could be because there are many more shopping centers compared to residential areas for example. This is really good in terms of which areas are safer, because it tells you what areas you are more likely to experience a crime, but it is likely exaggerated when you are in a residential area in that zip code. This is known as the denominator effect.

```
# --- 6. INDIVIDUAL PLOTS WITH NA LABELS ---
options(scipen = 999)

# Global Theme for consistency
map_theme <- theme_void() +
  theme(
    plot.title = element_text(face = "bold", size = 18, hjust = 0.5),
    legend.position = "right",
    plot.margin = margin(10, 10, 10, 10)
  )

# Function to generate the map to keep logic identical for all three
generate_crime_map <- function(cat_name, fill_high, title_text) {

  # Filter crime data for just this specific category
  category_data <- crime_summary %>% filter(category == cat_name)

  # Join with the map base so "Empty" Zips stay in the table
  plot_data <- map_base %>%
    left_join(category_data, by = c("match_id" = "zip_clean")) %>%
    mutate(crime_rate = (total_incidents / population) * 1000)

  ggplot() +
    # 1. The Map Polygons (Black outlines, White for NA areas)
    geom_sf(data = plot_data, aes(fill = crime_rate), color = "black", size = 0.1) +

    # 2. The NA Text (Only in zips with zero reported incidents)
    geom_sf_text(data = filter(plot_data, is.na(total_incidents)),
      aes(label = "NA"), size = 1.8, color = "grey60") +

    # 3. Highways & Shields (Standardized for all maps)
    geom_sf(data = dallas_highways, aes(size = road_type), color = "#ff9800", alpha = 0.5) +
    geom_point(data = highway_shields, aes(x = long, y = lat),
      color = "white", size = 7, shape = 21, fill = "white") +
    geom_image(data = highway_shields, aes(x = long, y = lat, image = image), size = 0.045) +

    # 4. Color Scale with Log Transformation
    scale_fill_gradient(
      low = "#f0f9ff",
      high = fill_high,
      na.value = "white",
      name = "Rate per 1k",

      # This is the "Log" part the professor asked for

```

```

    trans = "log10",

    # This keeps the scale "the same" (readable numbers instead of scientific notation)
    breaks = c(0.1, 1, 10, 100, 1000),
    labels = c("0.1", "1", "10", "100", "1000")
  ) +

  scale_size_manual(values = c("Interstate" = 0.7, "Tollway" = 0.4, "US Highway" = 0.4), guide = "none") +
  coord_sf(xlim = c(-97.02, -96.58), ylim = c(32.62, 33.02), expand = FALSE) +
  map_theme + labs(title = title_text)
}

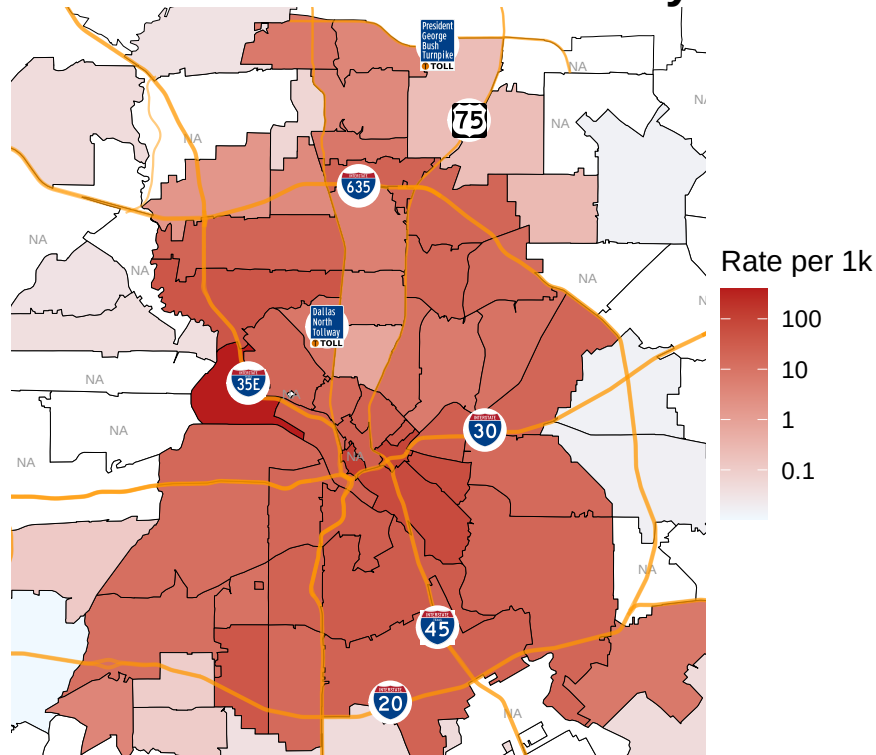
# GENERATE THE THREE PLOTS
# We use different colors: Red for Violent, Blue for Property, Green for Society
p1 <- generate_crime_map("Persons (Violent)", "#b71c1c", "Dallas: Violent Crime Density")
p2 <- generate_crime_map("Property", "#01579b", "Dallas: Property Crime Density")
p3 <- generate_crime_map("Society", "#1b5e20", "Dallas: Society Crime Density")

# Display them (In a PDF Knit, these will appear one after another)
p1

## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data

```

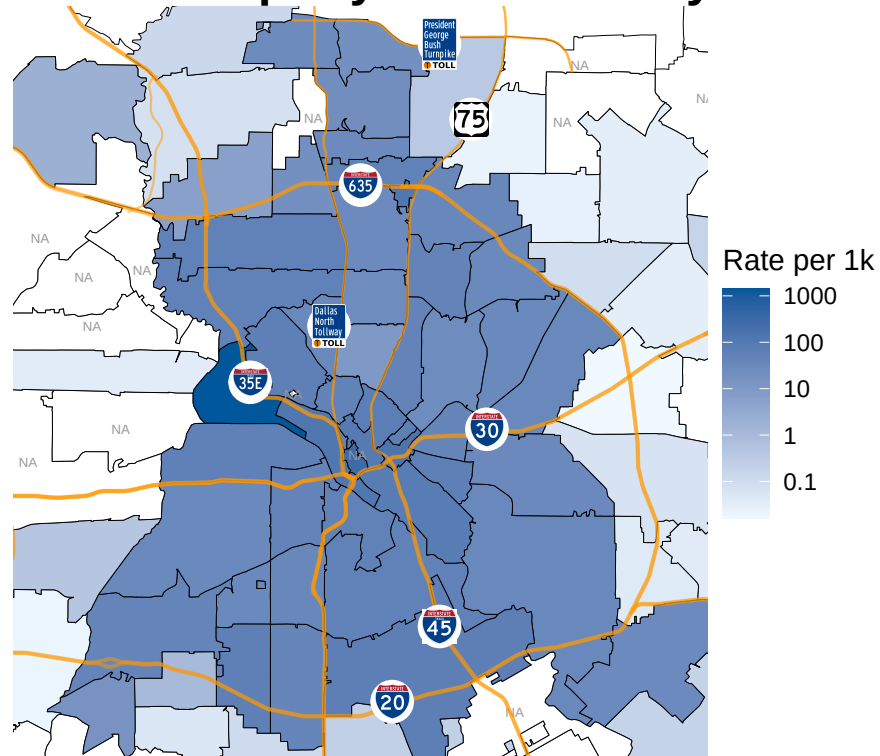
Dallas: Violent Crime Density



p2

```
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not  
## give correct results for longitude/latitude data
```

Dallas: Property Crime Density



p3

```
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not  
## give correct results for longitude/latitude data
```

Dallas: Society Crime Density

